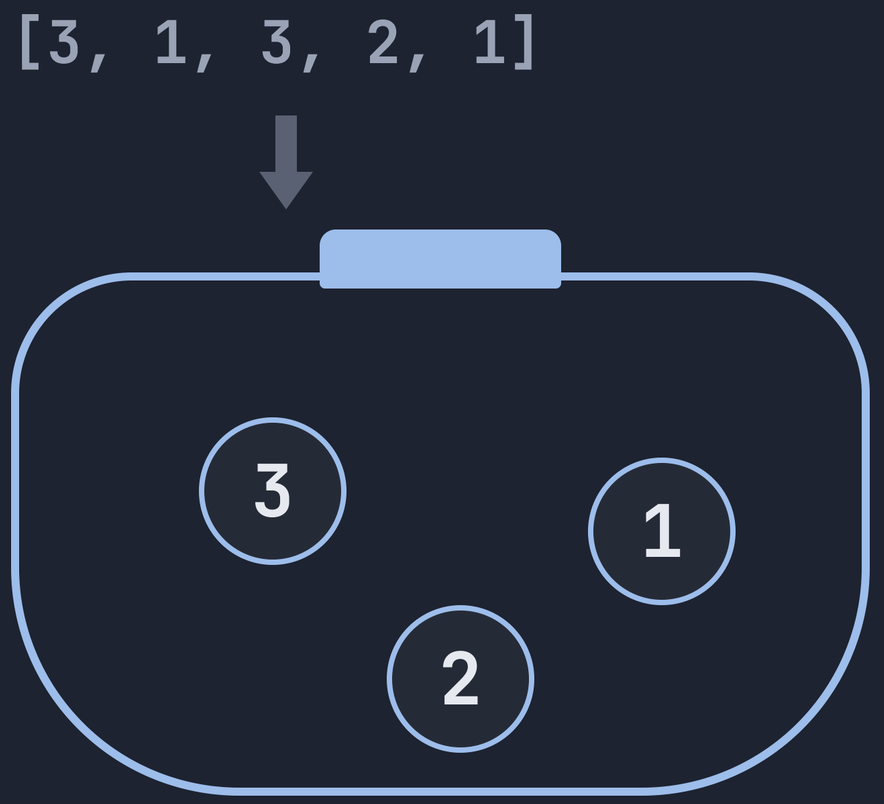
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 5
집합

중복 없이 모으고
겹치는 것 찾기



오늘의 순서

1 집합이 뭔가요 4–12쪽

2 바꾸고 확인하기 13–20쪽

3 집합 연산 21–26쪽

4 비교와 오류 27–29쪽

5 실습 30–33쪽

3쪽은 노트북 준비, 34쪽은 한 장 요약이에요.

들어가기

15에서 쓰던 노트북에 이어서 적어요

① VS Code로 hello-uv 폴더를 열고 이 파일을 열어요

② 커널이 .venv인지 봐요

basics.ipynb + Code + Markdown ▷ Run All ↻ Restart .venv (Python 3.14.6)

```
# 15 딕셔너리
menu = {"coffee": 4500, "latte": 5000}
# 16 집합  ← 여기부터 이어서 적어요
```

다시 쓰는 것

13	리스트와 튜플, `for`, `sorted()`, `count()`, f-string
14 · 15	`in`과 `not in`, 딕셔너리와 `KeyError`

1. 집합이 뭔가요

주문은 5잔인데, 메뉴는 몇 가지일까요

[1]
```python
orders = ["coffee", "latte", "coffee", "cake", "latte"]
print(orders)
print(len(orders))
```

Python

```
['coffee', 'latte', 'coffee', 'cake', 'latte']
5
```

← len은 잔 수를 세요. 메뉴 종류는 3가지인데요



같은 메뉴가 두 번씩 있어요. 겹친 것을 하나로 줄여야 종류가 나와요

같은 값을 하나만 남기는 그릇이 집합(set)이에요.

1. 집합이 뭔가요

집합은 네 번째 그릇이에요



	리스트 `[1, 2]`	튜플 `(1, 2)`	딕셔너리 `{"a": 1}`	집합 `{1, 2}`
순서	있어요	있어요	넣은 순서	없어요
중복	돼요	돼요	키는 안 돼요	안 돼요
꺼내기	[번호]	[번호]	[키]	못 꺼내요
바꾸기	돼요	안 돼요	돼요	돼요

집합은 딕셔너리처럼 중괄호를 쓰지만 키 없이 값만 적어요.

1. 집합이 뭔가요

set()에 리스트를 넣으면 같은 값이 하나만 남아요

```
[2]  nums = [3, 1, 3, 2, 1]
     print(nums)
     print(set(nums))
                                                    Python
```

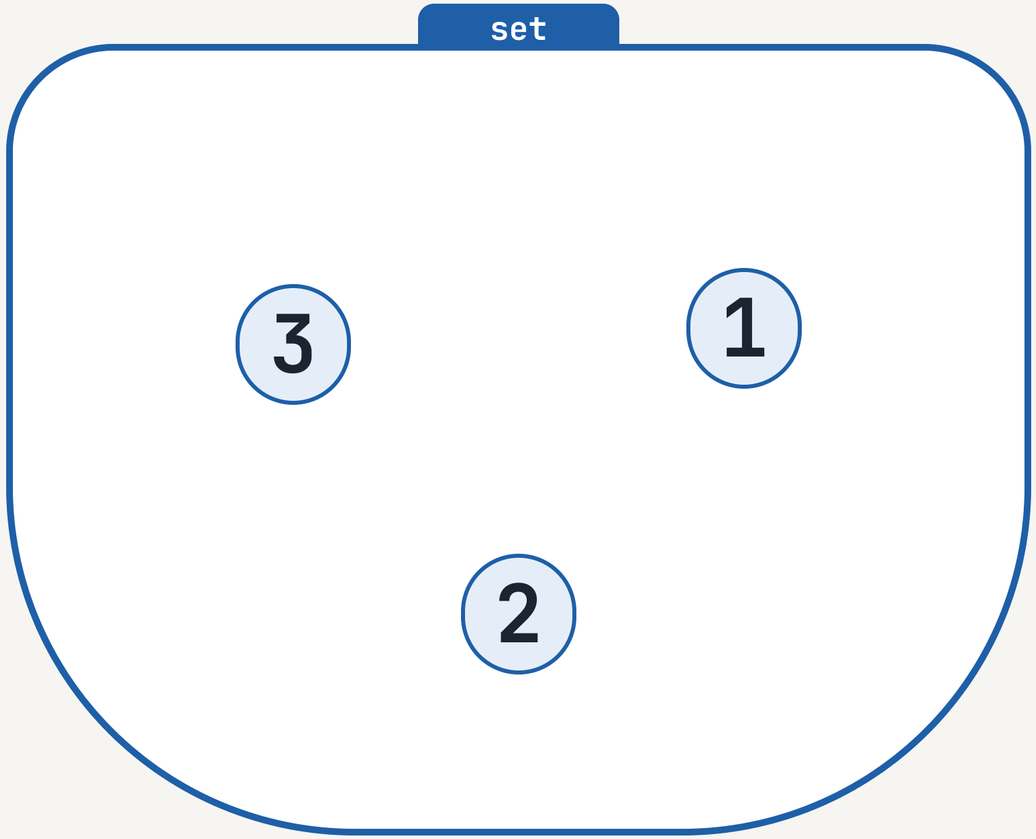
```
[3, 1, 3, 2, 1]
{1, 2, 3}   ← 3과 1이 하나씩만 남았어요
```

리스트 · 넣은 그대로, 칸마다 번호



겹친 값은 주머니에 또 들어가지 않아요

집합 · 같은 값은 하나만, 번호 없음

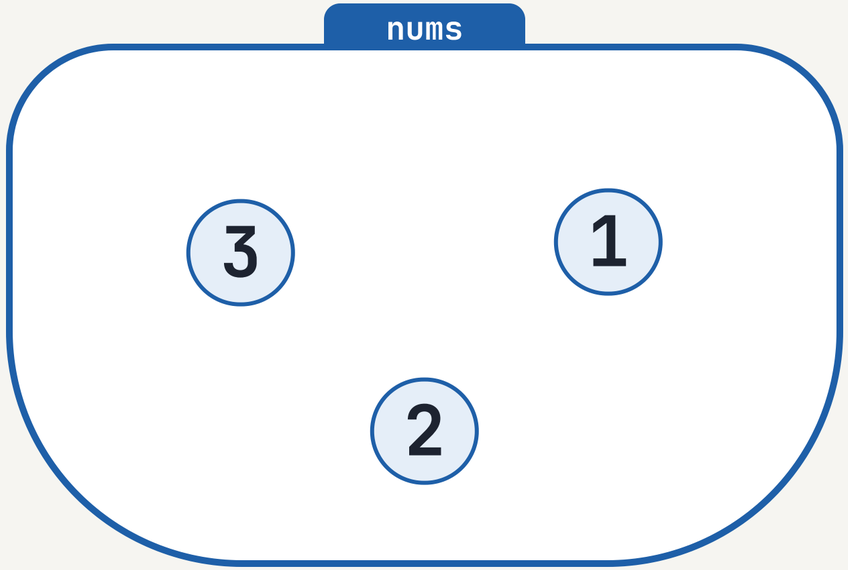


1. 집합이 뭔가요

중괄호 안에 값만 적으면 집합이에요

[3]
```python
nums = {3, 1, 2, 3}  ← 3을 두 번 적었어요
print(nums)
print(type(nums))
print(len(nums))
```

```
{1, 2, 3}          ← 오류 없이 하나만 남아요
<class 'set'>      ← set은 집합
3                  ← 값의 개수
```



중괄호 안을 보고 구분해요

`{"coffee": 4500}` 키: 값이 있으면 딕셔너리

`{"coffee", "latte"}` 값만 있으면 집합

1. 집합이 뭔가요

집합은 순서를 기억하지 않아요

처음 실행

```
[4] menus = {"coffee", "latte", "cake"}
    print(menus)
    print(sorted(menus))
```
Python

```
{'latte', 'cake', 'coffee'}
['cake', 'coffee', 'latte']
```

Restart 하고 이 셀만 다시 실행

```
[1] menus = {"coffee", "latte", "cake"}
    print(menus)
    print(sorted(menus))
```
Python

```
{'coffee', 'cake', 'latte'}
['cake', 'coffee', 'latte']
```

print(menus)는 실행할 때마다 순서가 바뀔 수 있어요

내 화면이 슬라이드와 달라도 틀린 게 아니에요.

sorted()에 넣으면 언제나 abc 순서 리스트예요

13에서 배운 그 sorted()예요. 결과는 리스트라 대괄호로 나와요.

1. 집합이 뭔가요

set(orders)로 메뉴 종류를 뽑아요

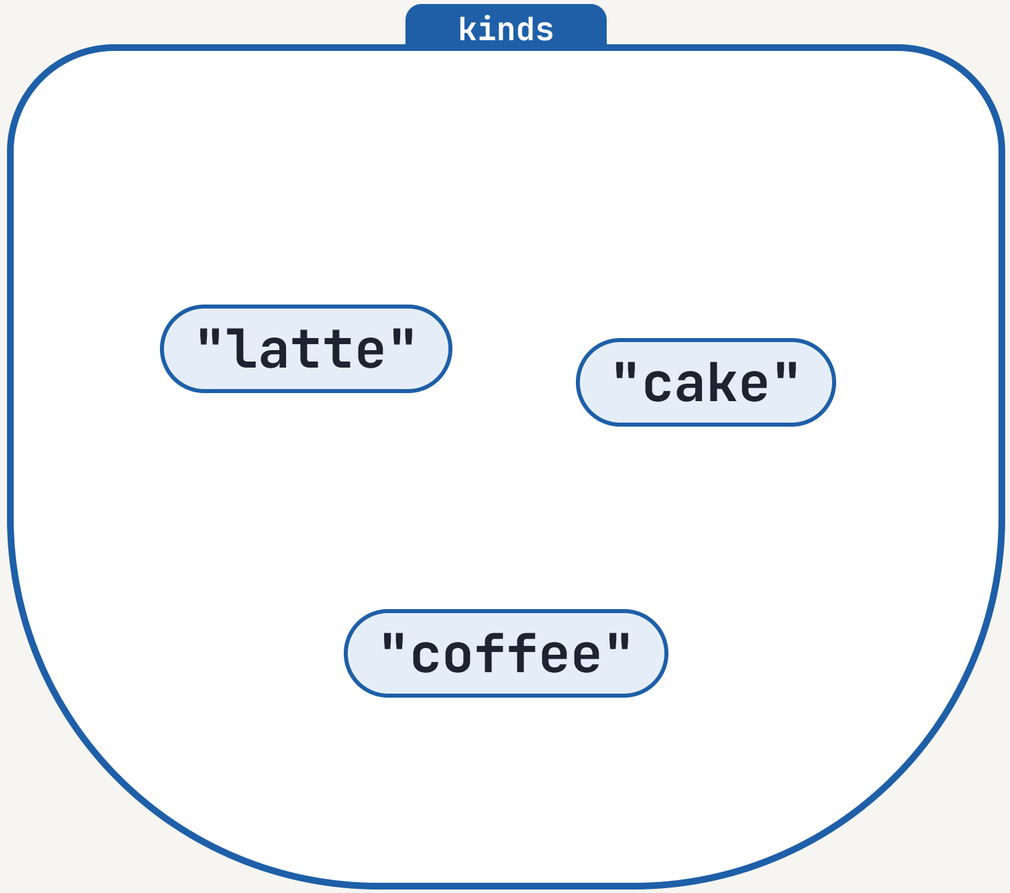
```
[5]  kinds = set(orders)
     print(kinds)
     print(len(orders), len(kinds))
     print(sorted(kinds))
```
Python

```
{'latte', 'cake', 'coffee'}
5 3   ← 주문 5잔, 메뉴 3가지
['cake', 'coffee', 'latte']   ← abc 순서
```

orders · 1파트 처음의 주문 5잔

"coffee" "latte" "coffee" "cake" "latte"

겹친 coffee, latte는 또 들어가지 않아요



1. 집합이 뭔가요

set(orders)는 for와 add를 한 줄로 줄인 거예요

줄여 쓰면 `kinds = set(orders)`

for와 add로 길게 쓰면

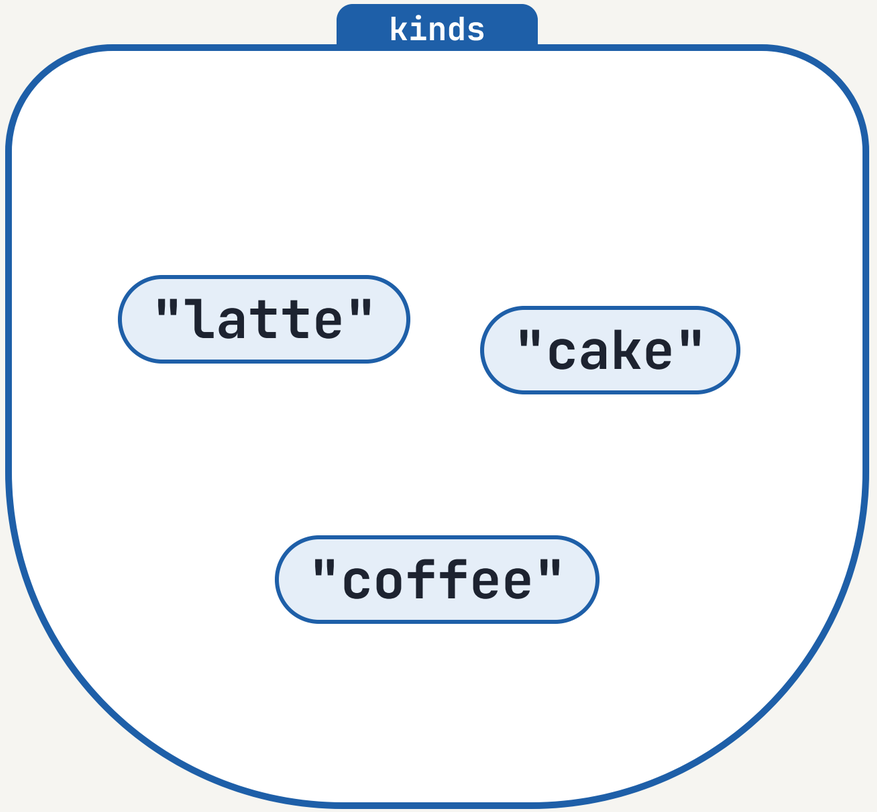
[6]
```
kinds = set()          ← 빈 주머니로 시작
for o in orders:       ← 주문을 하나씩 꺼내서
    kinds.add(o)       ← 넣어요. 있으면 그대로
print(sorted(kinds))
```
Python

```
['cake', 'coffee', 'latte']
```
← set(orders)와 같아요

참고 · 집합이 없으면 if는 17에서 배워요. 읽기만 해요.

```
kinds = []
for o in orders:
    if o not in kinds:
        kinds.append(o)
print(kinds)
```

```
['coffee', 'latte', 'cake']
```

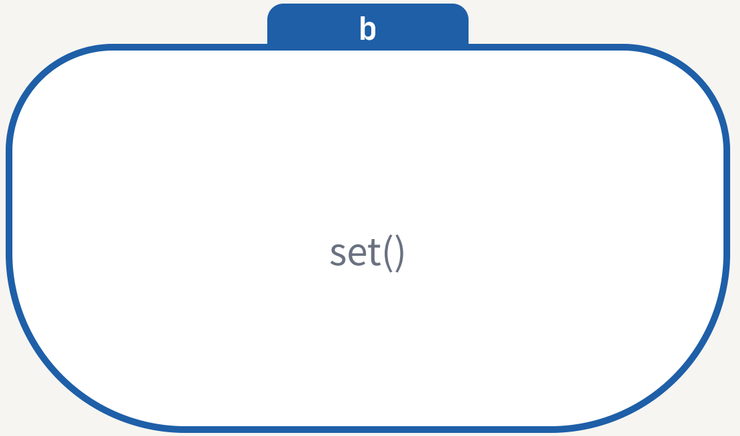


coffee, latte를 또 넣어도 하나씩

빈 집합은 {}가 아니라 set()이에요

[7]
```python
a = {}
print(type(a))
b = set()
print(type(b))
print(b)
print(len(b))
```

```
<class 'dict'>   ← {}는 15의 빈 딕셔너리예요
<class 'set'>    ← 빈 집합은 set()
set()            ← 찍어도 set()이라고 나와요
0
```



[8]
```python
cart = {}
cart.add("coffee")
```

```
AttributeError: 'dict' object has no attribute 'add'
```

× ~~cart = {}~~ → ✓ `cart = set()` add는 집합에 있어요

1. 집합이 뭔가요

숫자, 문자열, 튜플은 담고 리스트는 못 담아요

[9]
```python
ok = {1, "kim", (2, 3)}
print(len(ok))
```
Python

```
3
```

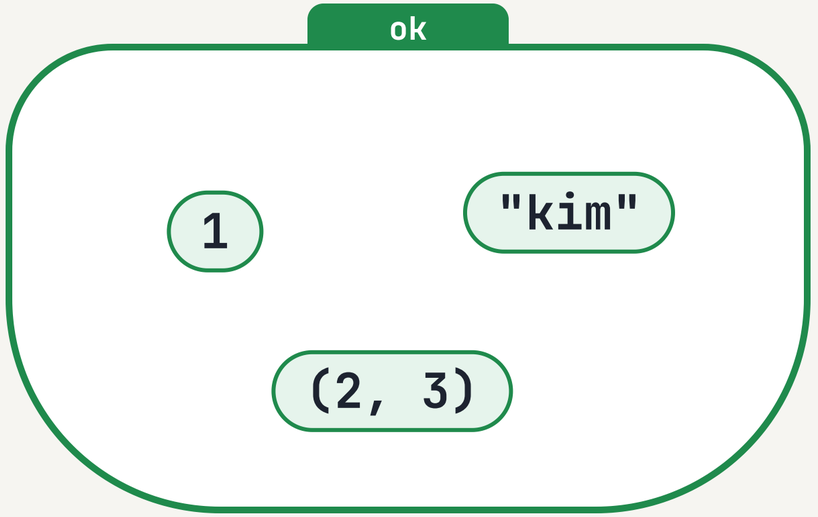
[10]
```python
bad = {[1, 2], 3}
```
Python

```
TypeError: unhashable type: 'list'
```

[11]
```python
d = {[1, 2]: "a"}
```
Python

```
TypeError: unhashable type: 'list'
```

15의 딕셔너리 키도 같은 규칙, 같은 오류예요



[1, 2] × 못 들어가요

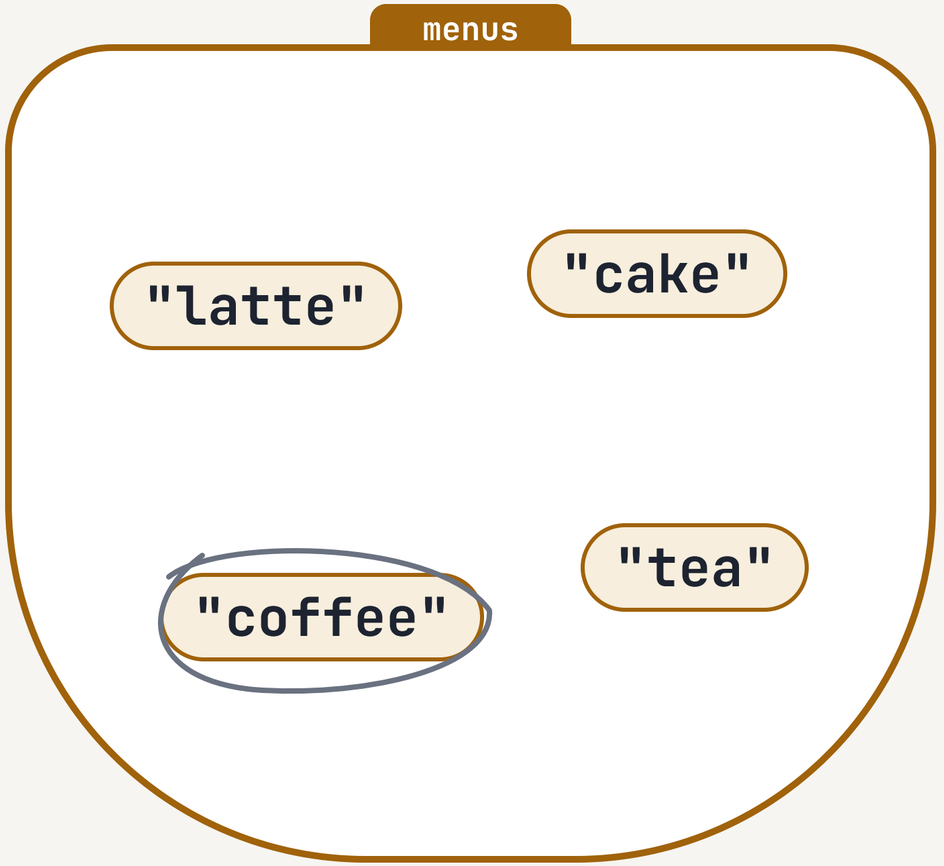
리스트는 append로 바뀔 수 있어서요

2. 바꾸고 확인하기

add로 하나 넣어요. 이미 있으면 그대로예요

[1]
```python
menus = {"coffee", "latte", "cake"}
menus.add("tea")
print(menus)
menus.add("coffee")  ← 이미 있어서 아무 일도 없어요
print(menus)
print(len(menus))
```

```
{'latte', 'cake', 'coffee', 'tea'}
{'latte', 'cake', 'coffee', 'tea'}
4  ← 4개 그대로
```



리스트는 append로 맨 뒤에 붙이지만,
집합은 자리가 없어서 그냥 넣어요.

2. 바꾸고 확인하기

remove는 없으면 오류, discard는 괜찮아요

[2]
```python
menus.remove("latte")    ← 있는 값을 빼요
print(menus)
```

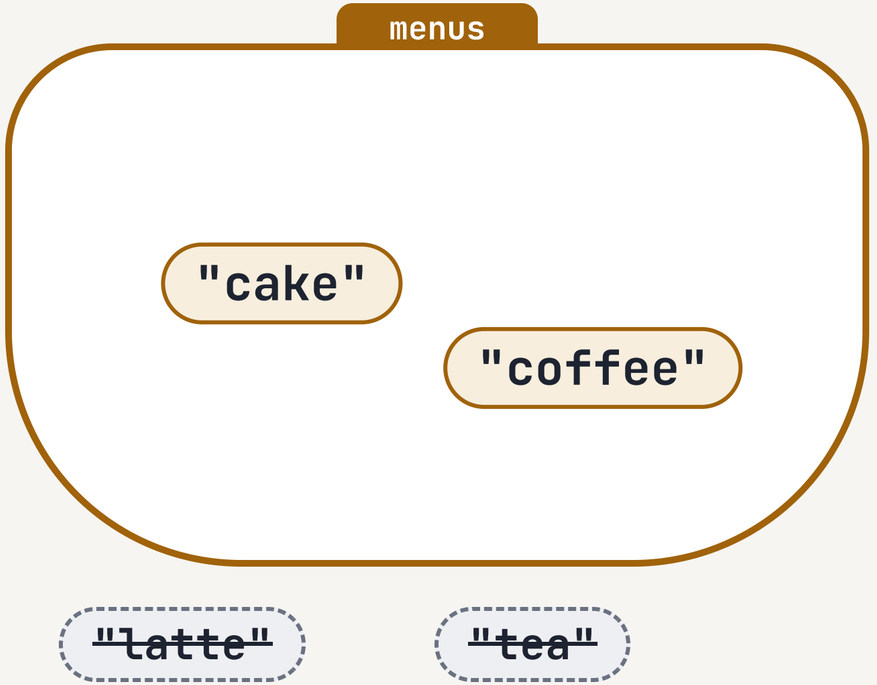
```
{'cake', 'coffee', 'tea'}
```

[3]
```python
menus.remove("juice")
```

```
KeyError: 'juice'    ← 없는 값이면 셀이 멈춰요
```

[4]
```python
menus.discard("juice")    ← 없으면 그냥 넘어가요
menus.discard("tea")      ← 있으면 빼요
print(menus)
```

```
{'cake', 'coffee'}
```



참고 · discard를 길게 쓰면 (if는 17에서 배워요)

```python
if "juice" in menus:
    menus.remove("juice")
```

15의 menu[키]와 get()처럼, remove와 discard도 짝이에요.

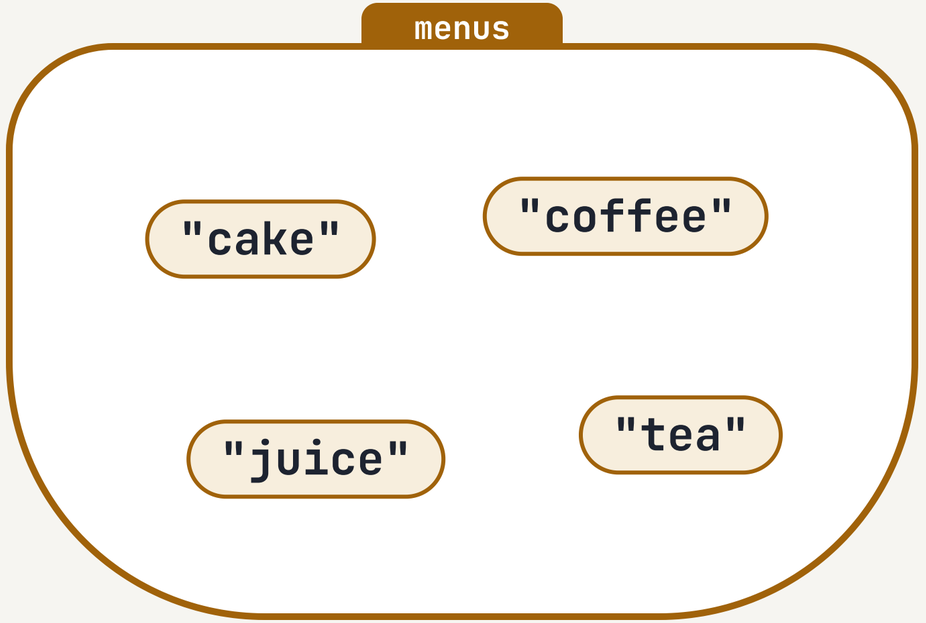
2. 바꾸고 확인하기

update로 여러 개를 한 번에 넣어요

[5]
```python
menus.update(["juice", "tea", "coffee"])
print(menus)
print(len(menus))
```

```
{'tea', 'cake', 'coffee', 'juice'}
4
```

← 2개 + 3개인데 4개예요



add는 값 하나, update는 리스트 안의 값을 모두 넣어요.
13의 리스트 extend와 비슷해요.

~~"coffee"~~ 이미 있어서 그대로

2. 바꾸고 확인하기

집합의 pop은 아무거나 하나 빼요

[6]
```python
one = menus.pop()
print(one)
print(menus)
nums = [10, 20, 30]
print(nums.pop())
```

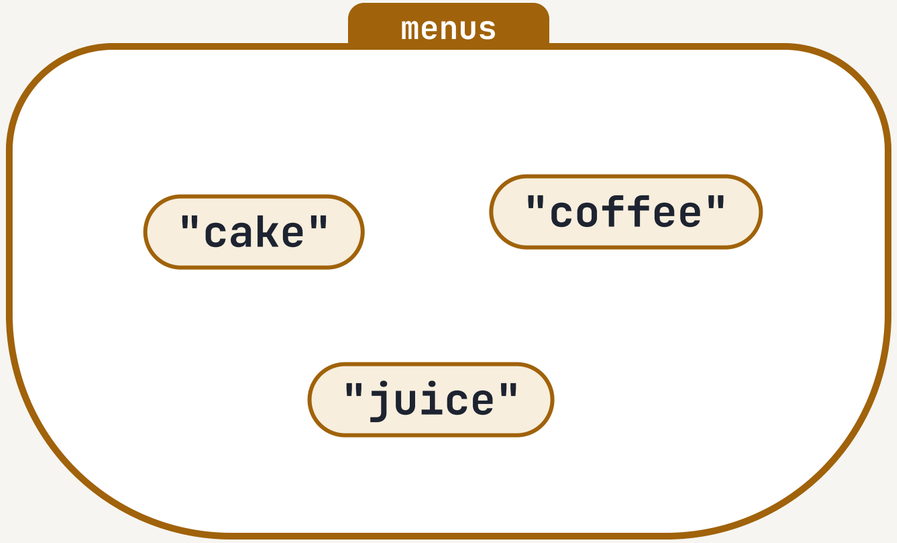
```
tea
{'cake', 'coffee', 'juice'}
30
```

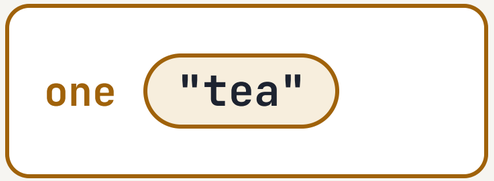
tea ← 어느 값이 나올지 정해져 있지 않아요

30 ← 리스트 pop은 맨 뒤를 빼요

[7]
```python
menus.clear()  ← 모두 비워요. 빈 주머니가 남아요
print(menus)
print(len(menus))
```

```
set()
0
```





2. 바꾸고 확인하기

in으로 들어 있는지 물어요

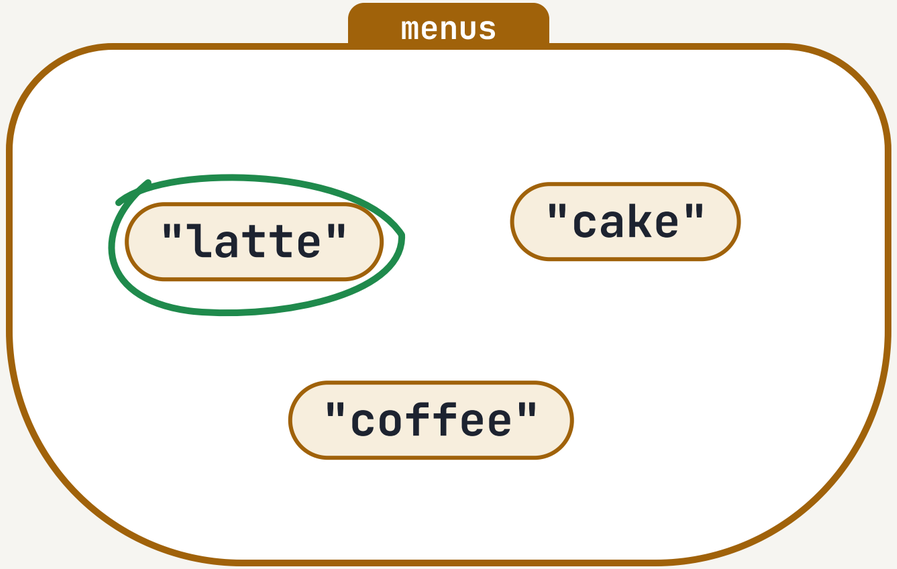
[8]
```python
menus = {"coffee", "latte", "cake"}
print("latte" in menus)
print("tea" in menus)
print("tea" not in menus)
```

```
True
False
True
```

← latte가 주머니에 있어요

← tea는 없어요

← 없는 게 맞아요



"tea" ← 밖에 있어요

14에서 리스트, 튜플, 문자열에 쓴 in 그대로예요.
집합은 꺼내기보다 들어 있는지 묻는 데 써요.

2. 바꾸고 확인하기

번호로 못 꺼내요. 필요하면 sorted()로 바꿔요

[9]
```python
menus[0]  ← 주머니에는 칸 번호가 없어요
```

```
TypeError: 'set' object is not subscriptable
```

[10]
```python
print(sorted(menus))
print(sorted(menus)[0])
print(list(menus))
```

```
['cake', 'coffee', 'latte']
cake  ← abc 순서 리스트라 [0]은 늘 cake
['latte', 'cake', 'coffee']  ← list()는 순서가 정해져 있지 않아요
```

✕ ~~menus[0]~~ → ✓ `sorted(menus)[0]` 15의 list(menu.keys())처럼 리스트로 바꿔요

2. 바꾸고 확인하기

집합에는 sort()가 없어요. sorted()를 써요

[11]
```
menus.sort()   ← sort()는 리스트에만 있어요
```
Python

```
AttributeError: 'set' object has no attribute 'sort'
```

[12]
```
nums = {30, 10, 20}
print(sorted(nums))
print(sorted(nums, reverse=True))
print(sorted(menus, reverse=True))
print(type(sorted(menus)))
print(menus)
```
Python

```
[10, 20, 30]   ← 작은 것부터
[30, 20, 10]   ← reverse=True면 큰 것부터
['latte', 'coffee', 'cake']
<class 'list'>   ← 결과는 새 리스트
{'latte', 'cake', 'coffee'}   ← menus는 그대로 집합
```

가장 많이 쓰는 모양

[13]
```
orders = ["tea", "cake", "tea",
          "coffee", "cake"]
print(sorted(set(orders)))
```
Python

```
['cake', 'coffee', 'tea']
```

겹친 주문을 없애고
abc 순서로 줄 세운 목록이
한 줄에 나와요

2. 바꾸고 확인하기

for로 하나씩 꺼낼 수 있어요

[14]
```python
for m in menus:
    print(m)
```

```
latte
cake
coffee
```

← 순서는 정해져 있지 않아요

[15]
```python
for m in sorted(menus):
    print(m)
```

```
cake
coffee
latte
```

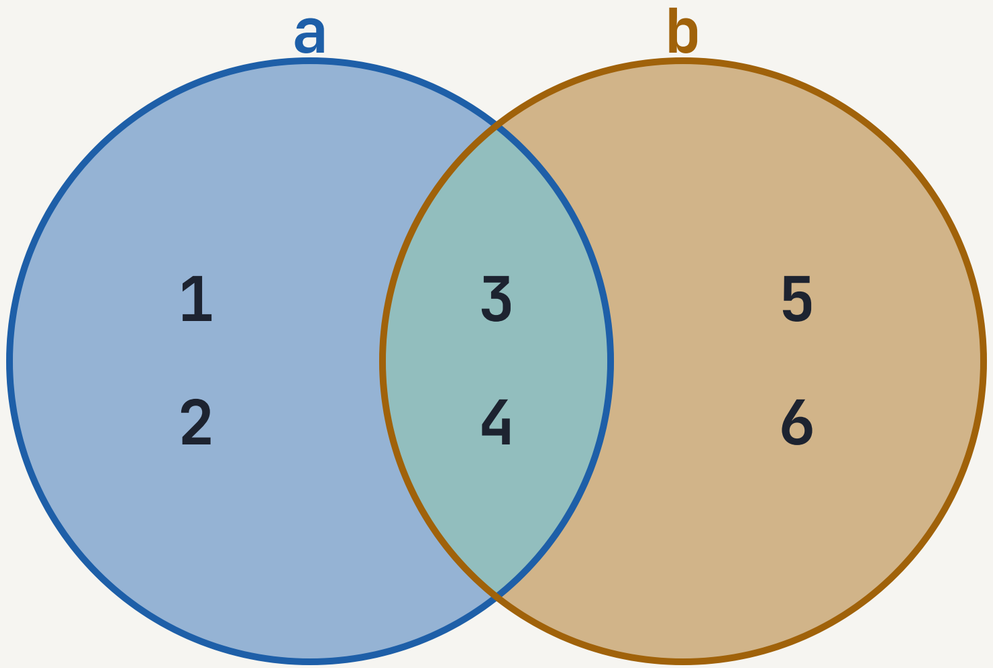
← abc 순서

`for 이름 in 집합:` 13의 리스트 for와 같은 모양이에요. 값마다 한 번씩 돌아요.

3. 집합 연산

두 집합을 원 두 개로 그려요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



a에만 있는 값 1, 2

a와 b에 다 있는 값 3, 4

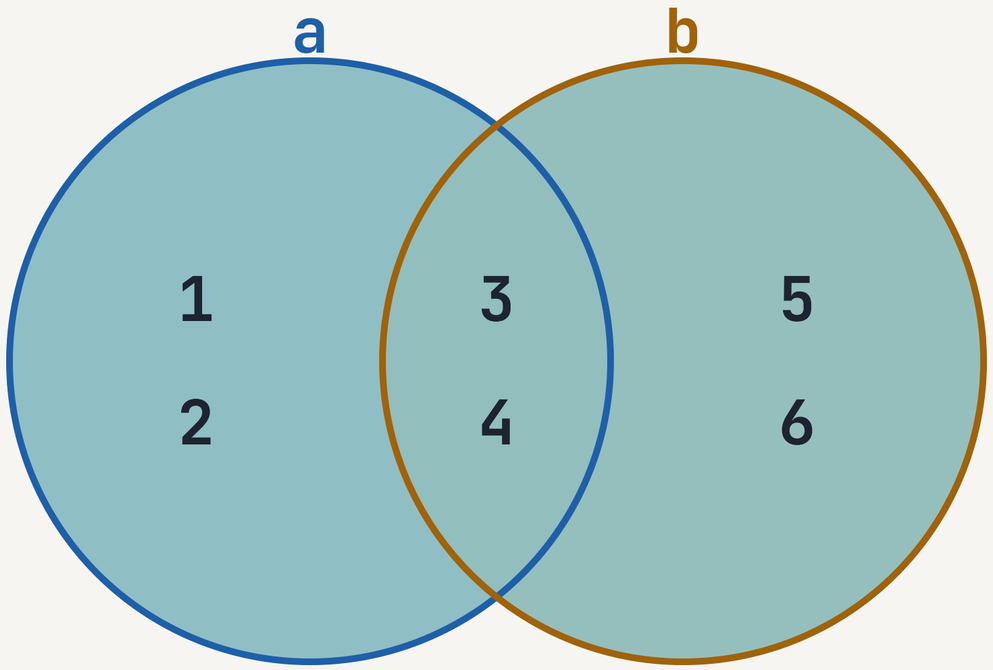
b에만 있는 값 5, 6

이 그림을 벤 다이어그램이라고 불러요.

3. 집합 연산

합집합 | 은 어느 쪽이든 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[1]
```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)
print(a.union(b))
```

```
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
```

← 겹친 3, 4는 하나씩

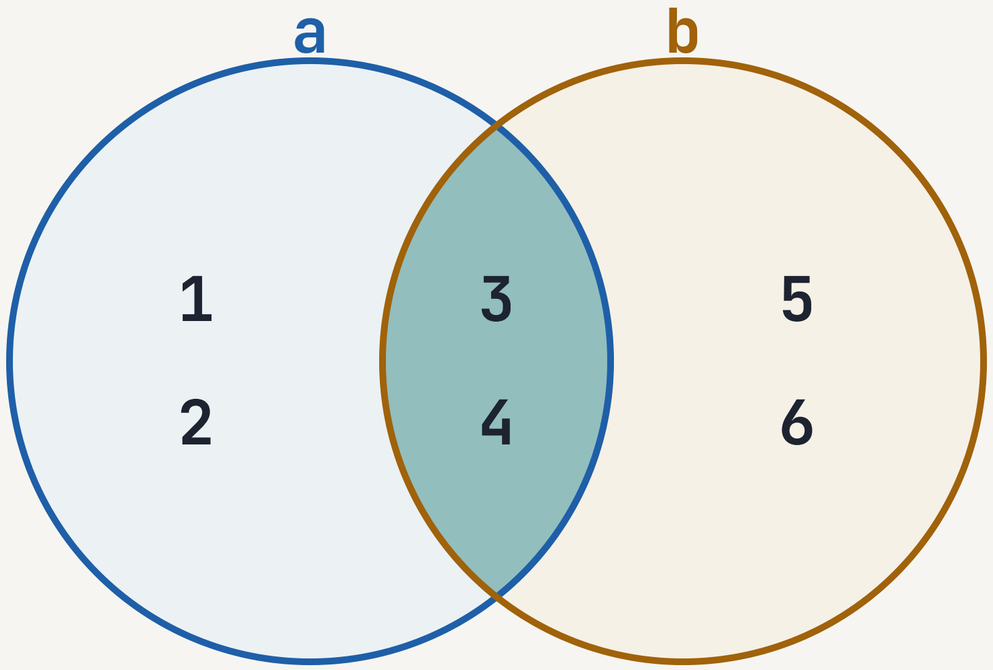
`a | b` 또는 `a.union(b)`

| 는 세로줄이에요. 보통 Enter 위 키를 Shift와 함께 눌러요.

3. 집합 연산

교집합 & 는 둘 다 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[2]
```python
print(a & b)
print(a.intersection(b))
```

```
{3, 4}  ← 겹친 곳만
{3, 4}
```

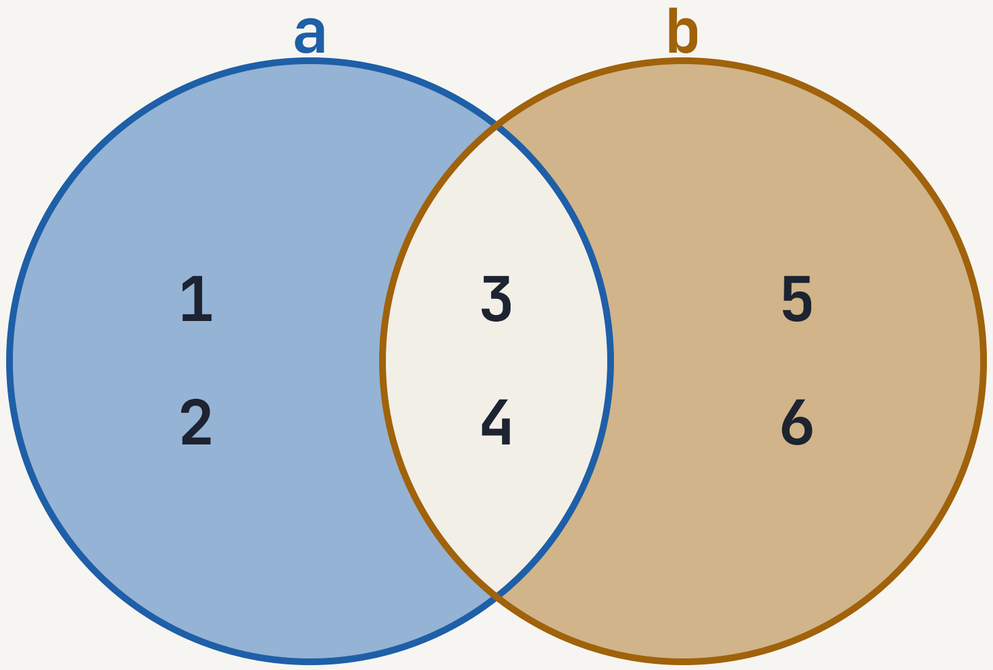
`a & b` 또는 `a.intersection(b)`

겹치는 사람, 겹치는 메뉴를 찾을 때 써요.

3. 집합 연산

차집합 - 는 앞에만 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[3]
```python
print(a - b)
print(b - a)
print(a.difference(b))
```

```
{1, 2}
{5, 6}
{1, 2}
```

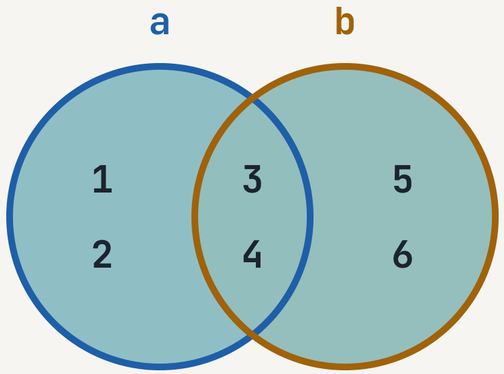
← a에서 b에 있는 걸 빼요

← 앞뒤를 바꾸면 결과도 달라요

`a - b` 또는 `a.difference(b)`

3. 집합 연산

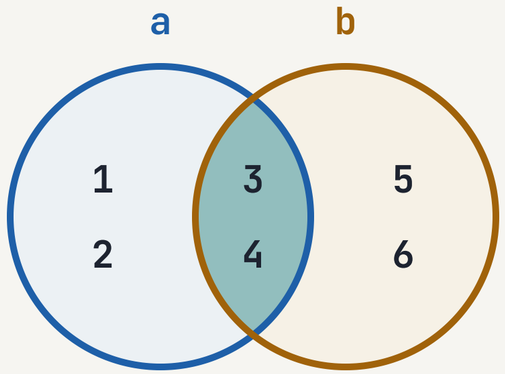
세 연산을 한 장에 모아요



`a | b` 어느 쪽이든

`a.union(b)`

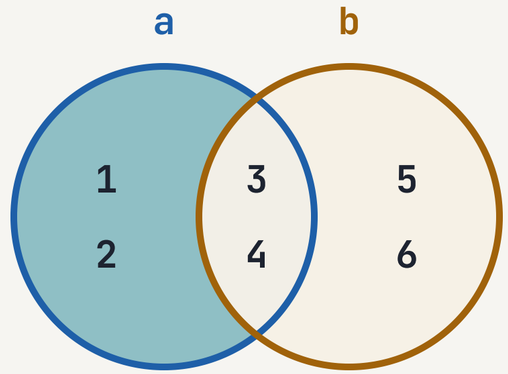
{1, 2, 3, 4, 5, 6}



`a & b` 둘 다

`a.intersection(b)`

{3, 4}



`a - b` a에만

`a.difference(b)`

{1, 2}

[4]
```python
c = a | b
print(a, b)
print(len(c), len(a & b))
```

```
{1, 2, 3, 4} {3, 4, 5, 6}
6 2
```

← 연산을 해도 a, b는 그대로예요

← 결과는 c처럼 변수에 담아요

3. 집합 연산

연산 기호도 for를 줄인 거예요

`a | b` 를 for와 add로 길게 쓰면

[5]
```python
c = set(a)      ← a를 복사해 담고
for x in b:     ← b를 하나씩 꺼내서
    c.add(x)    ← 넣어요. 겹친 3, 4는 그대로
print(c)
```

```
{1, 2, 3, 4, 5, 6}
```

& 와 - 는 b에 있는지 물어서 담을지 정해요.
그때 if가 필요해서 오른쪽은 읽기만 해요.

실제로 쓸 때는 기호 한 줄로 적어요.

참고 · a & b if는 17에서 배워요. 읽기만 해요.

```python
both = set()
for x in a:
    if x in b:
        both.add(x)
print(both)
```

```
{3, 4}
```

참고 · a - b if는 17에서 배워요. 읽기만 해요.

```python
only_a = set()
for x in a:
    if x not in b:
        only_a.add(x)
print(only_a)
```

```
{1, 2}
```

4. 비교와 오류

무엇에 담을지 이렇게 골라요

그릇	모양	언제	예
리스트	`[ ]`	**순서대로 쌓고 번호로 꺼낼 때**	하루 주문 orders
튜플	`( )`	**바뀌지 않는 묶음**	(메뉴, 가격, 수량) 한 줄
딕셔너리	`{키: 값}`	**이름으로 값을 찾을 때**	메뉴판 menu
집합	`{값}`	**중복 없애기, 있는지 묻기, 겹침 찾기**	팔린 메뉴 종류, 두 반 명단

섞어 써도 돼요. 5파트에서는 주문을 리스트로 쌓고, 비교할 때만 집합으로 바꿔요.

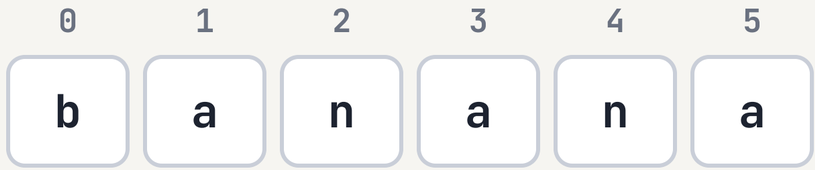
set("banana")는 글자로 쪼개져요

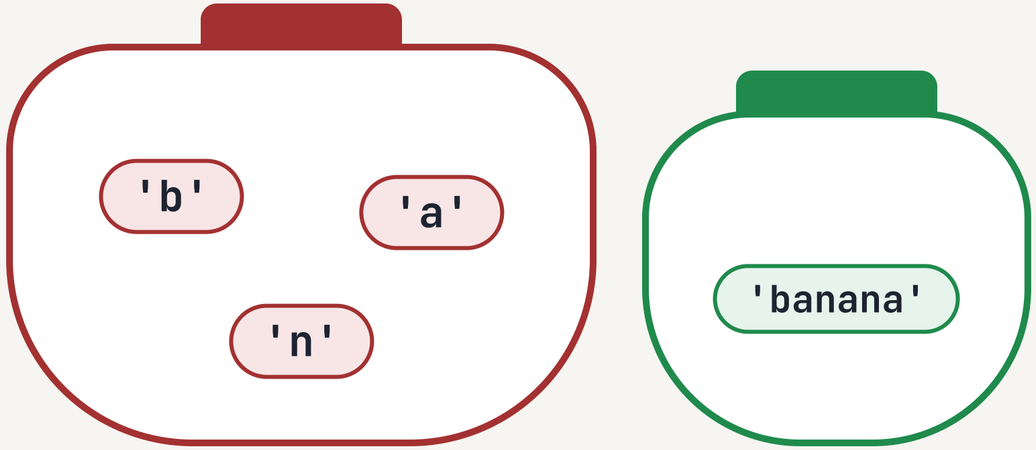
```
[1] print(set("banana"))
    print(len(set("banana")))
    print(set(["banana"]))
    print({"banana"})
```
Python

```
{'b', 'a', 'n'}
3
{'banana'}
{'banana'}
```

← 단어가 아니라 글자가 들어갔어요

← 리스트로 감싸거나 중괄호에 바로 적어요





13에서 문자열도 번호로 한 글자씩 꺼냈어요. 그래서 글자마다 하나씩 들어가요.

4. 비교와 오류

막히면 오류의 마지막 줄을 봐요

마지막 줄	까닭	고치기
`AttributeError: 'dict' object has no attribute 'add'`	빈 집합을 {}로 만들었어요	✓ `cart = set()`
`TypeError: 'set' object is not subscriptable`	집합에 번호를 붙였어요	✓ `sorted(menus)[0]`
`KeyError: 'juice'`	없는 값을 remove했어요	✓ `discard("juice")`
`TypeError: unhashable type: 'list'`	집합에 리스트를 넣었어요	✓ **튜플로 바꾸기**
`{'b', 'a', 'n'}`	set()에 문자열을 넣었어요	✓ `{"banana"}`
print 순서가 슬라이드와 달라요	집합은 순서가 없어요	✓ `sorted(menus)`

마지막 두 줄은 오류가 안 나지만 결과가 생각과 달라요.

5. 실습

하루 주문에서 팔린 메뉴 종류를 뽑아요

[1]

```python
orders = ["coffee", "latte", "coffee", "cake",
          "coffee", "latte", "tea"]
kinds = set(orders)                                  ① 겹친 주문을 하나로
print(f"주문 {len(orders)}잔, 메뉴 {len(kinds)}가지")   ② len으로 몇 가지인지
for m in sorted(kinds):                              ③ abc 순서로 돌기
    print(m, orders.count(m))                        ④ 잔 수는 원래 리스트에서 count
```

Python

```
주문 7잔, 메뉴 4가지
cake 1
coffee 3
latte 2
tea 1
```

← 집합은 중복을 없앴으니 셀 수 없어요

5. 실습

두 반을 다 듣는 사람과 한쪽만 듣는 사람

[2]
```python
python_class = {"kim", "lee", "park", "choi"}
data_class = {"park", "choi", "jung", "kang"}
both = python_class & data_class
print(sorted(both))
print(sorted(python_class - data_class))
print(sorted(data_class - python_class))
print(f"모두 {len(python_class | data_class)}명")
```
Python

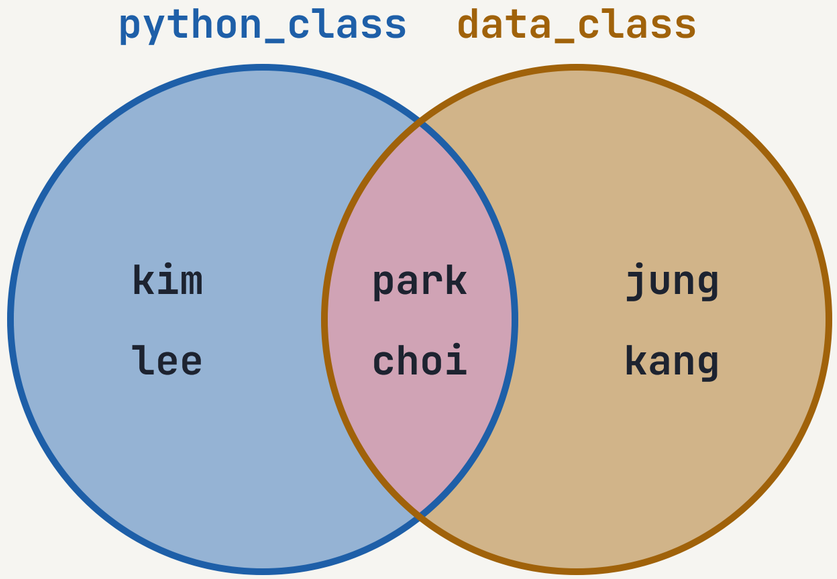
```
['choi', 'park']
['kim', 'lee']
['jung', 'kang']
모두 6명
```

← & 두 반 다

← - 파이썬반만

← - 데이터반만

← | 겹친 둘은 한 번만



8명을 적었지만 모두 6명

5. 실습

메뉴판에 없는 주문을 찾아요

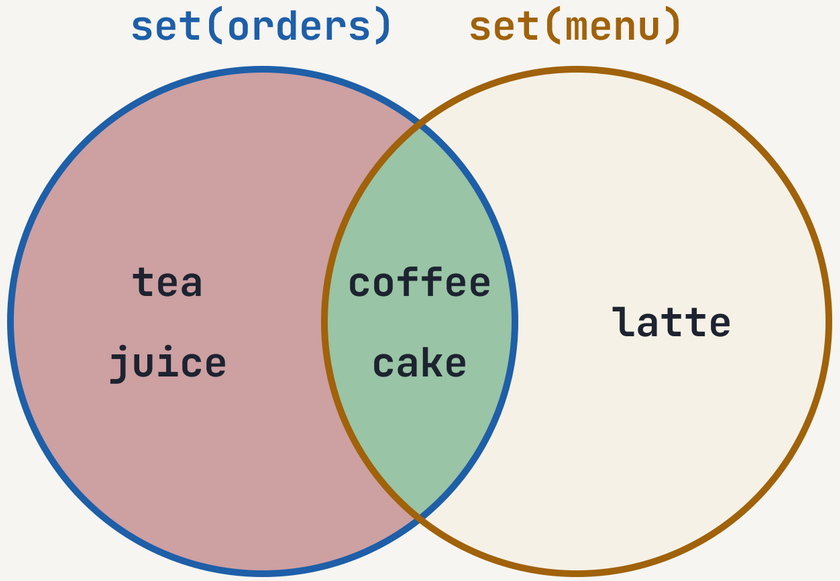
[3]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
orders = ["coffee", "tea", "cake", "juice", "coffee"]
print(set(menu))
print(sorted(set(orders) - set(menu)))
print(sorted(set(orders) & set(menu)))
```

Python

```
{'latte', 'cake', 'coffee'}
```
← 딕셔너리를 넣으면 키만 모여요

```
['juice', 'tea']
```
← 메뉴판에 없는 주문

```
['cake', 'coffee']
```
← 팔 수 있는 주문



15에서 sorted(menu)가 키만 줄 세운 것과 같아요.
latte는 메뉴판에만 있고 아무도 안 시켰어요.

5. 실습

팔 수 있는 주문만 계산해요

[4]
```python
total = 0
for m in sorted(set(orders) & set(menu)):          ← 메뉴판에 있는 주문만 돌아요
    qty = orders.count(m)                          ← 13의 count로 잔 수
    print(f"{m:<8}{qty}개 {menu[m] * qty:>7,}원")   ← 15처럼 키로 가격을 꺼내요
    total += menu[m] * qty
print(f"합계 {total:,}원")
```

Python

```
cake     1개    6,500원
coffee   2개    9,000원
합 계  15,500원
```

← tea, juice는 & 에서 빠져서 menu[m]에서 KeyError가 안 나요

15에서는 menu.get(name, 0)으로 없는 메뉴를 0원 처리했어요. 집합으로 먼저 거르면 그럴 일이 없어요.

한 장 요약

`{1, 2, 3}`	값만 적어 만들기	`s.discard("tea")`	있으면 빼기
`set(orders)`	리스트에서 중복 없애기	`"tea" in s`	들어 있는지 묻기
`set()`	빈 집합	`a | b  a & b  a - b`	합집합, 교집합, 차집합
`s.add("tea")`	하나 넣기	`sorted(s)`	줄 세운 리스트로

집합은 같은 값을 하나만 담는 주머니예요. 순서와 번호가 없어서 화면에 찍을 땐 sorted()를 써요.

17에서 if를 배우면 이번 챕터의 '참고' 코드를 직접 쳐 봐요.